

LAB3 - REPORT

Martí Gallego Giménez

u251445

Mateu Soria Waha

u251380

1. Introducció

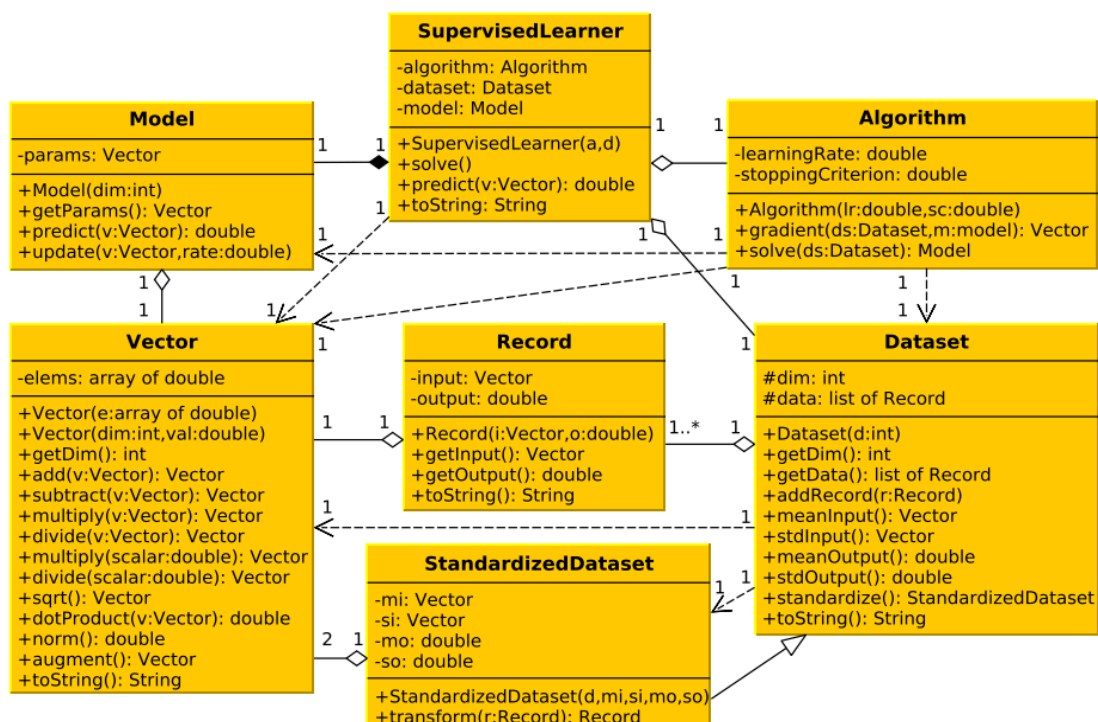
Aquest Lab, a part de les classes anteriors (Vector, Record, Dataset i StandardizedDataset) s'han implementat les classes **Model**, **Algorithm** i **SupervisedLearner**, reutilitzant les classes del laboratori anterior.

- La classe **Model** representa el vector de paràmetres del model de regressió i ofereix els mètodes per fer prediccions i actualitzar aquests paràmetres.
- La classe **Algorithm** implementa el descens de gradient: calcula el gradient de la funció de pèrdua i actualitza el model iterativament fins a convergir segons un criteri d'aturada.
- Finalment, la classe **SupervisedLearner** integra dataset, model i algorithm, encarregant-se d'entrenar el model i proporcionar prediccions a partir de vectors d'entrada.

2. Descripció de la solució

El Lab s'ha implementat dins de la carpeta anomenada Lab3 dins del nostre repositori de [Github](#). Conté, a part dels arxius del Lab1 i Lab2, tres nous arxius; Model.java, Algorithm.java i TestLearner.java en el qual es prova el codi i totes les seves implementacions. Els resultats de les proves es mostren per la terminal.

Les relacions entre les diferents classes es la següent:



2.1. Classe Model

Aquesta classe representa el model de regressió lineal mitjançant un vector de paràmetres θ (theta).

Atributs:

- *Vector params*: Vector que emmagatzema els paràmetres del model. Té dimensió $d+1$, on d és la dimensionalitat dels vectors d'entrada (el $+1$ correspon al terme bias).

Constructor:

- *Model(int dimension)*: Inicialitza el vector de paràmetres amb dimensió especificada, tots els valors inicialitzats a 0.

Mètodes:

- *getParams()*: Retorna el vector de paràmetres actual del model.
- *predict(Vector v)*: Calcula i retorna la predicció per a un vector d'entrada v ja augmentat. La predicció es calcula mitjançant el producte escalar entre els paràmetres del model i el vector d'entrada: $\theta \cdot v$.
- *update(Vector v, double rate)*: Actualitza els paràmetres del model aplicant la regla del descens de gradient: $\theta = \theta - \text{rate} \cdot v$, on v és el gradient i rate és la taxa d'aprenentatge.

2.2. Classe Algorithm

Aquesta classe implementa l'algorisme de descens de gradient per entrenar models de regressió lineal.

Atributs:

- *double learningRate*: Taxa d'aprenentatge (α) que controla la mida del pas en cada iteració del descens de gradient.
- *double stoppingCriterion*: Criteri d'aturada que determina quan el gradient és prou petit per finalitzar l'entrenament (per exemple, 10^{-6}).
- *int maxIterations*: Nombre màxim d'iteracions permeses per evitar bucles infinits (fixat a 10000).

Constructor:

- *Algorithm(double lr, double sc)*: Inicialitza la taxa d'aprenentatge i el criteri d'aturada amb els valors proporcionats.

Mètodes:

- *gradient(Dataset ds, Model m)*: Calcula el gradient de la funció de pèrdua MSE (Mean Squared Error) per al model actual donat el dataset. El gradient s'obté mitjançant la fórmula: $\nabla L(\theta) = (2/n) \sum (\hat{y}_i - y_i) \cdot x_i$ on n és el nombre de mostres, $\hat{y}_i$ és la predicció, y_i és el valor real i x_i és el vector d'entrada augmentat. El mètode recorre tots els registres del dataset, calcula l'error de predicció per a cadascun, i acumula els gradients. Finalment, divideix per n i multiplica per 2 segons la derivada del MSE.

- *solve(Dataset ds)*: Implementa el bucle principal del descens de gradient. Inicialitza un model nou amb tots els paràmetres a 0, calcula el gradient inicial i itera aplicant l'actualització $\theta = \theta - \alpha * \nabla L(\theta)$ fins que la norma del gradient sigui menor que el criteri d'aturada o s'arribi al màxim d'iteracions. Retorna el model entrenat.

2.3. Classe SupervisedLearner

Aquesta classe gestiona tot el procés d'aprenentatge supervisat integrant el dataset, l'algorisme i el model.

Atributs:

- *Algorithm algorithm*: Algorisme d'aprenentatge utilitzat per entrenar el model.
- *Dataset dataset*: Conjunt de dades d'entrenament amb els vectors d'entrada i sortides reals.
- *Model model*: Model entrenat, inicialment null fins que s'executa el mètode solve().

Constructor:

- *SupervisedLearner(Algorithm a, Dataset d)*: Inicialitza el SupervisedLearner amb un algorisme i un dataset. El model es manté null fins que s'entrena.

Mètodes:

- *solve()*: Assigna l'entrenament del model a l'algorisme cridant el mètode solve() d'Algorithm, i emmagatzema el model resultant.
- *predict(Vector v)*: Rep un vector d'entrada (sense augmentar), l'augmenta afegint el bias, i retorna la predicció del model entrenat utilitzant el mètode predict() de Model.
- *toString()*: Retorna una representació en format text del SupervisedLearner que inclou: el nom de la classe de l'algorisme utilitzat, la dimensió i nombre de mostres del dataset, i els paràmetres del model si ja ha estat entrenat (o indica que no està entrenat en cas contrari).

2.4. Classe TestLearner

Conté el mètode main() on es proven totes les funcionalitats implementades. Primer es crea un Dataset amb dimensió 1 i s'afegeixen quatre Records amb la relació lineal $y = 2x + 2$. A continuació, es crea un Algorithm amb taxa d'aprenentatge 0.1 i criteri d'aturada 10^{-6} . Després es crea una instància de *SupervisedLearner* amb l'algorisme i el dataset, i s'entrena el model cridant solve(). Finalment, es mostra la informació del learner per pantalla i es fa una predicció per al valor $x = 4.0$, esperant obtenir aproximadament $y = 10.0$.

3. Conclusions

Hem aconseguit implementar correctament l'algoritme de descens de gradient per a regressió lineal, integrant totes les classes desenvolupades als laboratoris anteriors.

La part més complexa va ser la implementació del mètode gradient() a la classe Algorithm, doncs es necessitava entendre bé la derivada de la funció de pèrdua MSE i com aplicarla correctament sobre tot el dataset. També, s'ha hagut de tenir molta cura en l'augment dels vectors per incloure el terme bias en les operacions de predicció.

Així, vam observar que la taxa d'aprenentatge té un impacte significatiu en la convergència del model, valors massa grans poden fer que l'algorisme no convergeixi. Per valors massa petits, l'entrenament seria molt lent. El criteri d'aturada també és crucial per a determinar quan el model ha après prou. Aquest laboratori ens ha permès consolidar conceptes fonamentals i bàsics de machine learning, com el descens de gradient i la regressió lineal.